

Reviewer Pack — Secant Variety Dimension Bounds Disjointness Communication C...

Entry 31bbf877f01c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 02:09:27 UTC

Secant Variety Dimension Bounds Disjointness Communication Complexity

Entry ID: 31bbf877f01c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 02:09:27 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry of Secant Varieties **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For any $n \times n$ matrix M representing the DISJOINTNESS function, the dimension of the secant variety of the variety defined by M 's rank-1 tensors is $\Omega(n)$.

Rationale (proposer's reasoning):

Secant variety dimensions capture the minimal number of rank-1 tensors needed to approximate a matrix, which correlates with communication complexity via tensor rank lower bounds. This bridges algebraic geometry's intrinsic geometric complexity to combinatorial communication protocols.

Taxonomy category: COMM_DISJ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1f39ff84aa51b175

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def transpose(A):
        n = len(A)
        T = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                T[j][i] = A[i][j]
        return T

    def rank_1_decomposition(M, iterations=50):
        n = len(M)
        U = [[random.random() for _ in range(n)] for _ in range(n)]
        V = [[random.random() for _ in range(n)] for _ in range(n)]

        for _ in range(iterations):
            U = matrix_multiply(U, M)
            V = matrix_multiply(V, transpose(M))

            U_norm = sum(sum(x**2 for x in row) for row in U)**0.5
            V_norm = sum(sum(x**2 for x in col) for col in V)**0.5

            U = [[x / U_norm for x in row] for row in U]
            V = [[x / V_norm for x in col] for col in V]

        return U, V
```

```

def border_rank(M):
    n = len(M)
    rank = 1
    while True:
        U, V = rank_1_decomposition(M, iterations=50)
        M_approx = matrix_multiply(U, transpose(V))
        if max(max(abs(x) for x in row) for row in (M - M_approx)) < 1e-6:
            return rank
        rank += 1

def secant_dimension(M):
    n = len(M)
    rank = border_rank(M)
    return rank * (rank + 1) // 2

n = random.randint(5, 40)
M = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

secant_dim = secant_dimension(M)
ratio = secant_dim / n

return {
    "metric_name": "secant_dimension_ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": ratio >= 1,
    "counterexample": "" if ratio >= 1 else f"n={n}, M={M}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = r["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fac7a877.py", line 90, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fac7a877.py", line 73, in run_trial
    secant_dim = secant_dimension(M)
                 ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fac7a877.py", line 67, in secant_dimension
    rank = border_rank(M)
           ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fac7a877.py", line 61, in border_rank
    if max(max(abs(x) for x in row) for row in (M - M_approx)) < 1e-6:
           ^^^^^^^^^^^^^
```

TypeError: unsupported operand type(s) for -: 'list' and 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in matrix subtraction; no data collected to assess conjecture
| next: Fix matrix subtraction implementation to handle list-of-lists structures properly

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109139
2	preregistration	ollama_remote	qwen3:8b	0	0	31937
3	novelty	ollama_remote	qwen3:8b	0	0	27682
4	novelty	ollama_remote	qwen3:8b	0	0	34979
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11069
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9210
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12474
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9813
9	judge	ollama_remote	qwen3:8b	0	0	23009

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 269313 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/31bbf877f01c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/31bbf877f01c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/31bbf877f01c.tar.gz (if generated)